Assignment 1

General Instructions

- This is an **individual assignment**. All submitted work must be your own.
- **Plagiarism in any form is strictly prohibited** and will result in failure.
- The assignment is graded as **Pass (G) / Fail (U)**.
- To pass, **both Question 1 and Question 2 must be completed in full**.
- **The report must be written in a single-column format.**
- **Each question must be answered in a clearly labeled section corresponding to the question number.**
- **Answers that are missing will be considered incomplete.** .

Submission Requirements

- Submit a **single ZIP file** via Canvas.
- The ZIP file must contain:
  - One PDF report
  - All implementation files (source code)
- **Do not include the dataset** in your submission.
- ZIP file naming format:

`firstname_lastname.zip` (e.g., `john_doe.zip`)

Question 1: Optimization Using Gradient Descent

Consider the following function with two parameters x and y.

$$f(x,y)=(x-3)^2+(y+2)^2$$

Tasks

a) Compute the partial derivative of the function with respect to x.

b) Compute the partial derivative of the function with respect to y.

c) Implement gradient descent **from scratch** using Python and find the minimum of the function.

d) Plot the gradient descent results (e.g., loss curve or optimization trajectory).

Requirements

- All mathematical derivations must be shown clearly in the report.

Submission

- The report must include derivations, explanations, plots, and final results.
- Code must be submitted as separate files inside the ZIP archive.

Question 2: Deep Learning for Image Classification

In this question, you will design a custom deep learning model and compare it against established architectures on the same task.

Dataset

- CIFAR-100 dataset
- Available at: `https://www.cs.toronto.edu/~kriz/cifar.html`

Models to Implement

1. A **custom convolutional neural network (CNN)** designed by you
   - Must contain between **4 and 8 convolutional layers**
   - Must be trained **from scratch**
2. The following pre-defined architectures:
   - ResNet-50
   - VGG-19
   - DenseNet-121
   - EfficientNet (B0 or B1)
3. For the pre-defined architectures, **pre-trained weights may be used**.

You may use **PyTorch, TensorFlow, Keras**.

Report Requirements

Your report must contain the following sections:

1. **Custom Architecture Description**
   - Layer-by-layer description or architecture diagram
   - Number of layers, filter sizes, strides, and padding
2. **Training Configuration**
   - Activation functions
   - Loss function
   - Optimizer
   - Regularization methods (e.g., dropout, batch normalization)
   - Hyperparameters (learning rate, batch size, epochs) with justification
3. **Data Preprocessing**
   - Normalization
   - Resizing (if applicable)
   - Data augmentation
   - Grayscale conversion (if applicable)
4. **Training Analysis**

- Plots per epoch:
  - Training loss
  - Validation loss
  - Training accuracy
  - Validation accuracy
- Discussion of overfitting or underfitting
- Description of mitigation strategies (e.g., early stopping)

5. **Weight Evolution Analysis**
   - Visualize 1–2 convolutional filters from the first layer:
     - Before training
     - After training
   - Explain observed changes

6. **Evaluation Metrics**
   - Accuracy
   - True Positives (TP)
   - True Negatives (TN)
   - False Positives (FP)
   - False Negatives (FN)
   - F1-score
   - Results must be presented in a single table

7. **Model Comparison**
   - Compare the custom CNN with ResNet-50, VGG-19, DenseNet-121, and EfficientNet
   - Discuss performance differences and possible reasons

Code Requirements

- Code must be readable, and executable.
- All reported results must be reproducible using the submitted code.